

Ticket Manager

Luiz Eduardo Garzon de Oliveira

Sumário

1	Introdução	3
1.1	O problema	3
1.2	Levantamento de Requisitos	3
1.2.1	Manutenção de Funcionários (Cadastrar e Editar)	3
1.2.2	Cadastro de Ticket Entregue (Cadastrar e Editar)	4
1.2.3	Emissão de Relatório	4
2	Metodologia	5
2.1	Tecnologias Utilizadas	5
2.2	Arquitetura da Solução	5
2.2.1	Camada Apresentação	6
2.2.2	Camada Aplicação	6
2.2.3	Camada Domínio	7
2.2.4	Camada Infraestrutura	7
2.3	Roteiro de Desenvolvimento	7
2.4	Modelagem do Banco de Dados	8
2.4.1	Script de Criação do Banco de Dados	9
2.4.2	Diagrama Entidade-Relacionamento (ER)	10
2.4.3	Consulta para Visualização Combinada de Funcionários e Tickets	11
2.5	Resumo da Implementação	12
2.5.1	Resumo da Implementação da Camada Domínio	12
2.5.2	Resumo da Implementação da Camada Aplicação	13
2.5.3	Resumo da Implementação da Camada Infraestrutura	13
2.5.4	Resumo da Implementação da Camada Console	14
2.5.5	Executando o Programa	14
2.5.6	Configurando a String de Conexão	15
2.5.7	Executando Testes Automatizados	15
3	Casos de Uso	16
3.1	Caso de Uso: Cadastrar um Funcionário	17
3.2	Caso de Uso: Editar um Funcionário	17
3.3	Caso de Uso: Listar Funcionários	20
3.3.1	Listar Todos os Funcionários Ativos	20
3.3.2	Listar Todos os Funcionários Inativos	21
3.3.3	Listar Todos os Funcionários do Sistema	21
3.4	Caso de Uso: Registrar Tickets Entregues	22

3.5	Caso de Uso: Editar Tickets Entregues	23
3.6	Caso de Uso: Emitir Relatórios	24
3.6.1	Emitir Todos os Tickets de um Funcionário	24
3.6.2	Emitir Relatório Completo por Período	25
3.6.3	Emitir Relatório Completo de Todo o Sistema	25
3.7	Caso de Uso: Encerrar Programa	26
4	Considerações Finais e Próximos Passos	27
4.1	Próximos Passos	27
4.1.1	Desacoplamento das Funcionalidades de Edição	27
4.1.2	Filtros Personalizados para Relatórios	27
4.1.3	Interface Web	27

1

Introdução

1.1 O problema

O objetivo deste projeto é desenvolver um sistema de gestão de tickets de refeição que permita controlar quantos tickets foram entregues a cada funcionário, com consultas por período e cálculo do total geral entregue.

A justificativa para o desenvolvimento deste sistema é que a empresa fictícia utilizada como cenário possui mais de 1000 funcionários e, atualmente, o controle de tickets de refeição é feito por meio de um caderno físico — um processo manual, sujeito a erros e sem possibilidade de consultas estruturadas.

1.2 Levantamento de Requisitos

Antes de detalhar os requisitos de cada funcionalidade, duas regras gerais se aplicam a todo o sistema:

- Nenhum registro pode ser excluído — esta é uma regra inegociável do sistema;
- Toda desativação de um registro é lógica, feita por meio do campo *Situação*.

1.2.1 Manutenção de Funcionários (Cadastrar e Editar)

Id Identificador do funcionário.

- Campo obrigatório;
- Deve ser único no banco de dados.

Nome Contém o nome do funcionário.

- Campo obrigatório.

CPF Número do CPF do funcionário.

- Campo obrigatório;
- Deve ser único (não deve permitir dois funcionários com o mesmo CPF).

Situação Identifica se o cadastro está ativo ou inativo.

- Campo obrigatório;
- Deve receber apenas os valores A para Ativo e I para Inativo;
- Ao gravar pela primeira vez, não deve permitir gravar como I.

Data de Alteração Determina a última data e hora em que o cadastro foi alterado.

1.2.2 Cadastro de Ticket Entregue (Cadastrar e Editar)

Id Identificador do ticket.

- Campo obrigatório;
- Deve ser único no banco de dados.

Funcionário Identifica o funcionário ao qual o ticket pertence.

- Campo obrigatório.

Quantidade Quantidade de tickets entregues.

- Campo obrigatório.

Situação Identifica se o cadastro está ativo ou inativo.

- Campo obrigatório;
- Deve receber apenas os valores A para Ativo e I para Inativo;
- Ao gravar pela primeira vez, não deve permitir gravar como I.

Data Entrega Armazena a data e hora em que o ticket foi entregue.

- Campo obrigatório;
- Campo desabilitado para o usuário;
- Deve gravar a data/hora atual ao gravar o registro pela primeira vez.

1.2.3 Emissão de Relatório

O sistema deve oferecer alguma forma de o usuário identificar os tickets entregues por funcionário em um determinado período, bem como o total geral entregue naquele período.

2

Metodologia

Este capítulo descreve a metodologia utilizada no desenvolvimento da solução para o problema enunciado. O problema foi interpretado com o olhar “dividir para conquistar”, ou seja, quebrado em partes a fim de definir um fluxo estruturado de implementação, passo a passo, visando:

- Arquitetura desacoplada, pensando em escalabilidade, manutenção e rastreamento de erros;
- Documentação técnica construída ao longo do processo de desenvolvimento;
- Entendimento do escopo antes de codificar.

2.1 Tecnologias Utilizadas

O sistema foi desenvolvido utilizando as seguintes tecnologias e ferramentas:

- Linguagem C# (SDK do .NET 10.0.301);
- Banco de dados MySQL (gerenciado via MySQL Workbench);
- Editor de código Visual Studio Code, com a extensão C# Dev Kit.

2.2 Arquitetura da Solução

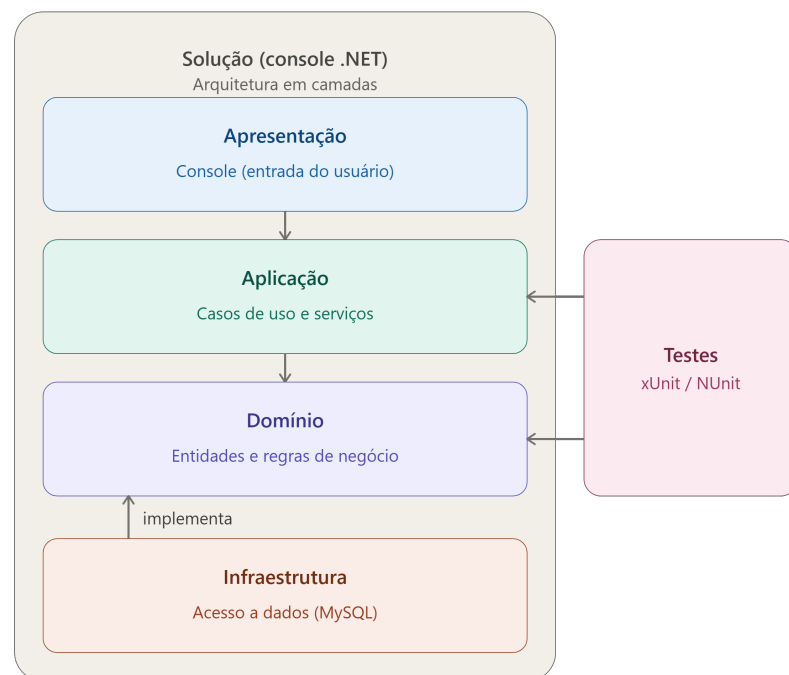
A arquitetura foi pensada de modo a dividir o problema em camadas que permitissem e facilitassem:

- Testabilidade;

- Manutenção;
- Rastreamento de erros;
- Escalabilidade.

A figura a seguir representa visualmente a forma como o sistema foi organizado em camadas.

Figura 1 – Arquitetura em camadas do sistema



2.2.1 Camada Apresentação

A camada de Apresentação (implementada como uma aplicação de console) tem como responsabilidade ler o que o usuário digita e exibir o que o sistema retorna. Não há regra de negócio, consultas SQL ou validação de CPF nesta camada (essas responsabilidades pertencem à Aplicação e ao Domínio).

2.2.2 Camada Aplicação

A camada de Aplicação orquestra os casos de uso do sistema (por exemplo, “registrar entrega de ticket”).

2.2.3 Camada Domínio

O Domínio concentra as entidades e as regras de negócio puras, sem qualquer conhecimento sobre banco de dados.

2.2.4 Camada Infraestrutura

A Infraestrutura implementa o acesso ao MySQL, seguindo as interfaces que o próprio Domínio define. É por isso que a seta de dependência entre essas camadas aponta de baixo para cima: quem depende de quem se inverte (*Dependency Inversion Principle*, ou DIP), isso facilita a testabilidade do código possibilitando validar a Aplicação e o Domínio sem depender de um banco de dados real.

2.3 Roteiro de Desenvolvimento

1. Modelagem do banco de dados (entidades, campos, tipos, *constraints*);
2. Esqueleto do projeto em C# com os cinco módulos da arquitetura (Domínio, Aplicação, Infraestrutura, Apresentação, Testes);
3. Implementação do Domínio:
 - a) Classes `Funcionario` e `TicketEntregue`;
 - b) Regras de negócio (CPF único, campo `Situacao` não pode nascer como I, etc.);
 - c) Interfaces de repositório;
4. Implementação da Infraestrutura:
 - a) Conexão com o MySQL e repositórios concretos;
5. Implementação da Aplicação:
 - a) Casos de uso (cadastrar funcionário, registrar ticket, gerar relatório);
6. Implementação da Apresentação:
 - a) Menu de console;
7. Testes unitários:
 - a) Regras de negócio do Domínio;
 - b) Casos de uso da Aplicação.

2.4 Modelagem do Banco de Dados

Optou-se por iniciar o desenvolvimento pela modelagem do banco de dados, pois isso facilita a compreensão do que o sistema precisará para ser implementado. Como a camada de Apresentação é responsável apenas por se comunicar com as camadas internas do sistema, modelar o banco de dados é, na prática, modelar a base sobre a qual todo o restante do sistema será construído.

Duas entidades nascem diretamente das regras de negócio:

- `funcionarios`;
- `tickets_entregues`.

Elas se relacionam por uma cardinalidade **1:N** — um funcionário pode ter vários tickets entregues.

Como o sistema nunca permite a exclusão de registros, o campo `Situação` é o único mecanismo de “desativação” que existe. A regra de que esse campo não pode “nascer como I” depende do momento da operação — inserção ou atualização — e por isso essa validação precisa ocorrer na camada de Domínio, como regra de negócio. O banco de dados garante apenas que o valor armazenado seja sempre A ou I.

2.4.1 Script de Criação do Banco de Dados

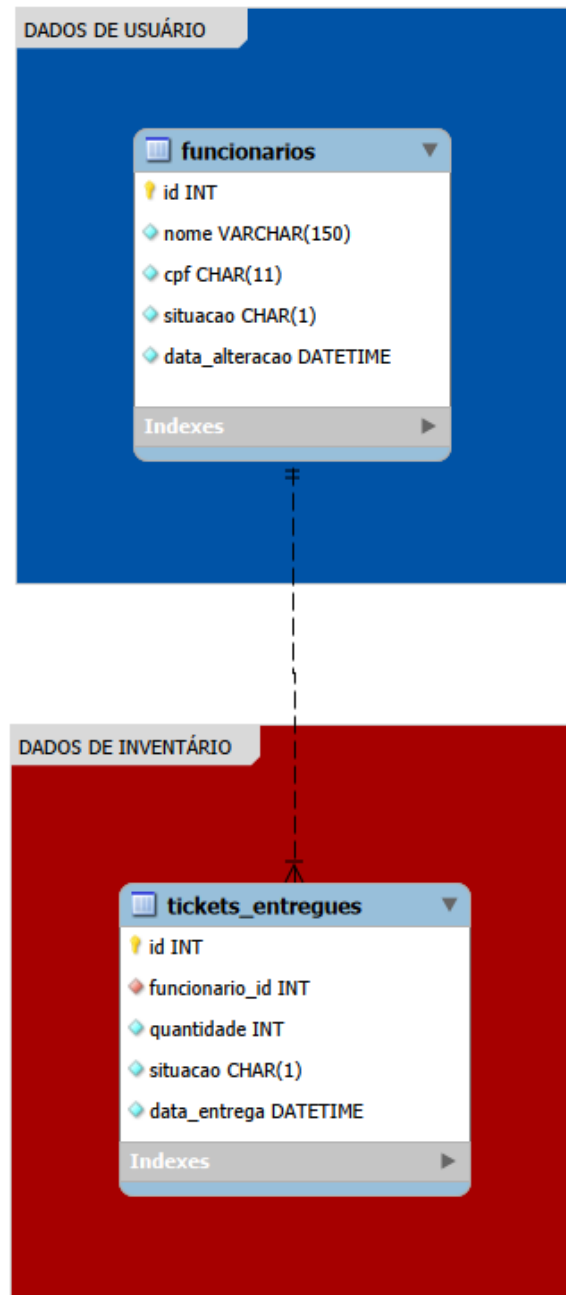
Código 1 – Script de criação do banco de dados e das tabelas

```
1  -- =====
2  -- Criacao do banco de dados
3  -- =====
4  CREATE DATABASE ticket_manager_db
5      CHARACTER SET utf8mb4
6      COLLATE utf8mb4_unicode_ci;
7
8  USE ticket_manager_db;
9
10 -- =====
11 -- Tabela de funcionarios, conforme o levantamento de requisitos
12 -- =====
13 CREATE TABLE funcionarios (
14     id                INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
15     nome              VARCHAR(150) NOT NULL,
16     cpf               CHAR(11) NOT NULL,
17     situacao          CHAR(1) NOT NULL DEFAULT 'A',
18     data_alteracao    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
19                     ON UPDATE CURRENT_TIMESTAMP,
20
21     CONSTRAINT uq_funcionarios_cpf UNIQUE (cpf),
22     CONSTRAINT ck_funcionarios_situacao CHECK (situacao IN ('A', 'I'))
23 ) ENGINE = InnoDB;
24
25 -- =====
26 -- Tabela de tickets entregues, conforme o levantamento de requisitos
27 -- =====
28 CREATE TABLE tickets_entregues (
29     id                INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
30     funcionario_id    INT UNSIGNED NOT NULL,
31     quantidade        INT UNSIGNED NOT NULL,
32     situacao          CHAR(1) NOT NULL DEFAULT 'A',
33     data_entrega      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
34
35     CONSTRAINT fk_tickets_funcionario
36         FOREIGN KEY (funcionario_id) REFERENCES funcionarios (id),
37     CONSTRAINT ck_tickets_situacao CHECK (situacao IN ('A', 'I')),
38     CONSTRAINT ck_tickets_quantidade CHECK (quantidade > 0)
39 ) ENGINE = InnoDB;
40
```

2.4.2 Diagrama Entidade-Relacionamento (ER)

A figura a seguir apresenta o diagrama entidade-relacionamento correspondente ao script anterior.

Figura 2 – Diagrama entidade-relacionamento do sistema



2.4.3 Consulta para Visualização Combinada de Funcionários e Tickets

Esta consulta permite visualizar as informações de cada funcionário (Id, nome e situação cadastral), o total de tickets a ele entregues e o detalhamento de cada ticket individual, separados pelo caractere |, no formato: Id: 1 (Qtd: 3, Situação: I) | Id: 6 (Qtd: 3, Situação: A).

Código 2 – Consulta SQL para visualização combinada de funcionários e tickets

```

1 SELECT
2     funcionarios.id AS funcionario_id,
3     funcionarios.nome,
4     funcionarios.situacao,
5     SUM(tickets_entregues.quantidade) AS total_tickets,
6     GROUP_CONCAT(
7         CONCAT(
8             'Id: ', tickets_entregues.id,
9             ' (Qtd: ', tickets_entregues.quantidade,
10            ', Situação: ', tickets_entregues.situacao,
11            ') '
12        )
13        ORDER BY tickets_entregues.id
14        SEPARATOR ' | '
15    ) AS tickets
16 FROM funcionarios
17 LEFT JOIN tickets_entregues
18     ON funcionarios.id = tickets_entregues.funcionario_id
19 GROUP BY
20     funcionarios.id,
21     funcionarios.nome,
22     funcionarios.situacao;
23

```

A figura a seguir apresenta o resultado da consulta anterior.

Figura 3 – Resultado da consulta combinada de funcionários e tickets

	funcionario_id	nome	situacao	total_tickets	tickets
▶	1	Luiz Eduardo	A	6	Id: 1 (Qtd: 3, Situação: I) Id: 6 (Qtd: 3, Situação: A)
	2	Júlio César	A	4	Id: 2 (Qtd: 4, Situação: A)
	3	Marco Aurélio	I	NULL	NULL
	4	Ragnar Lodbrok	A	2	Id: 3 (Qtd: 2, Situação: A)
	5	Michael Scott	A	7	Id: 4 (Qtd: 5, Situação: I) Id: 5 (Qtd: 2, Situação: A)

Fonte: O autor (2026)

2.5 Resumo da Implementação

A seguir, apresenta-se um resumo de como a implementação está organizada, detalhando o conteúdo de cada um dos quatro projetos que compõem a solução (conforme ilustrado na [Figura 1](#)).

A cadeia de dependência entre eles é a seguinte: Console depende de Application e de Infrastructure; Application depende apenas de Domain; Infrastructure depende apenas de Domain (implementando as interfaces que ele declara); e Domain não depende de nenhum dos outros três — é por isso que a seta de dependência sempre aponta *para* ele, nunca dele para fora.

Console → Application → Domain Infrastructure → Domain

2.5.1 Resumo da Implementação da Camada Domínio

O projeto `TicketManager.Domain` é o núcleo das regras de negócio do sistema e não depende de nenhum outro projeto da solução — é a base sobre a qual todos os demais se apoiam.

Entities/Funcionarios.cs

Define a classe `Funcionario`.

Entities/TicketEntregue.cs

Define a classe `TicketEntregue`.

Enums/SituacaoCadastro.cs

Define o enum com os valores `Ativo` e `Inativo`, compartilhado pelas duas entidades.

Exceptions/DomainException.cs

Define a exceção customizada utilizada em toda violação de regra de negócio.

Repositories/IFuncionarioRepository.cs

Interface com os métodos que um repositório de funcionário precisa implementar.

Repositories/ITicketEntregueRepository.cs

Interface equivalente para tickets entregues.

2.5.2 Resumo da Implementação da Camada Aplicação

O projeto `TicketManager.Application` contém os casos de uso do sistema, orquestrando as regras do Domínio com os repositórios. Depende apenas do `Domain` — utiliza suas entidades e interfaces de repositório, sem qualquer conhecimento sobre a existência do MySQL.

Services/FuncionarioService.cs

Implementa os métodos:

- `Cadastrar`, `Editar`, `Ativar`, `Inativar`;
- `ObterFuncionarioPorId`, `ObterFuncionarioPorCpf`;
- `ListarTodos`, `ListarTodosAtivos`, `ListarTodosInativos`.

Services/TicketEntregueService.cs

Implementa os métodos:

- `RegistrarEntrega`, `EditarQuantidade`;
- `Ativar`, `Inativar`, `ListarPorFuncionario`.

Services/RelatorioService.cs

Implementa os métodos `GerarRelatorioPorPeriodo` e `GerarRelatorioCompleto`.

Relatorios/ItemRelatorioFuncionario.cs

Record usado dentro do relatório por período — um item por funcionário.

Relatorios/RelatorioPeriodo.cs

Record que reúne a lista de `ItemRelatorioFuncionario` e o total geral.

Relatorios/RelatorioFuncionarioTicketsCompleto.cs

Record usado pelo relatório completo (funcionário e a lista de seus tickets).

2.5.3 Resumo da Implementação da Camada Infraestrutura

O projeto `TicketManager.Infrastructure` é onde o sistema efetivamente se comunica com o banco de dados MySQL. Assim como a Aplicação, depende apenas do `Domain` — implementa as interfaces de repositório declaradas ali, sem qualquer conhecimento sobre `Application` ou `Console`.

MySqlConnectionFactory.cs

Cria objetos `MySqlConnection` a partir de uma *string* de conexão.

Repositories/FuncionarioRepository.cs

Implementação concreta de `IFuncionarioRepository`, utilizando `Dapper`.

Repositories/FuncionarioRegistro.cs

DTO interno que espelha a tabela funcionarios, usado exclusivamente pelo Dapper.

Repositories/TicketEntregueRepository.cs

Implementação concreta de ITicketEntregueRepository.

Repositories/TicketEntregueRegistro.cs

DTO interno que espelha a tabela tickets_entregues.

2.5.4 Resumo da Implementação da Camada Console

O projeto `TicketManager.Console` é a porta de entrada do sistema, onde o usuário interage diretamente. É também o único projeto que depende tanto de `Application` quanto de `Infrastructure`: do primeiro, para chamar os casos de uso; do segundo, exclusivamente dentro do `Program.cs`, para montar os repositórios concretos.

Program.cs

É o *composition root* do sistema — o único ponto onde `Infrastructure` e `Application` se encontram. Monta a *string* de conexão, instancia a `MySQLConnectionFactory`, os dois repositórios concretos e os três serviços, e por fim cria o `MenuPrincipal`, chamando seu método `Executar`.

MenuPrincipal.cs

Implementa o menu interativo: o método `Executar`, o método `ExibirMenu` e um método privado para cada opção do menu:

- `CadastrarFuncionario`, `EditarFuncionario`;
- `ListarFuncionarios` (com suas três variações);
- `RegistrarTicket`, `EditarTicket`;
- `EmitirRelatorios` (com suas três variações).

2.5.5 Executando o Programa

Para executar o programa, basta acessar a pasta raiz do projeto e executar o seguinte comando no terminal:

```
dotnet run --project src/TicketManager.Console
```

2.5.6 Configurando a String de Conexão

Antes de executar o programa, é necessário configurar a string de conexão com o banco de dados em `src/TicketManager.Console/Program.cs`, informando as credenciais do próprio ambiente:

Código 3 – Configuração da string de conexão

```
1 var connectionString = "Server=localhost;Port=3306;Database=
2   ticket_manager_db;User=root;Password=SUA_SENHA_AQUI";
```

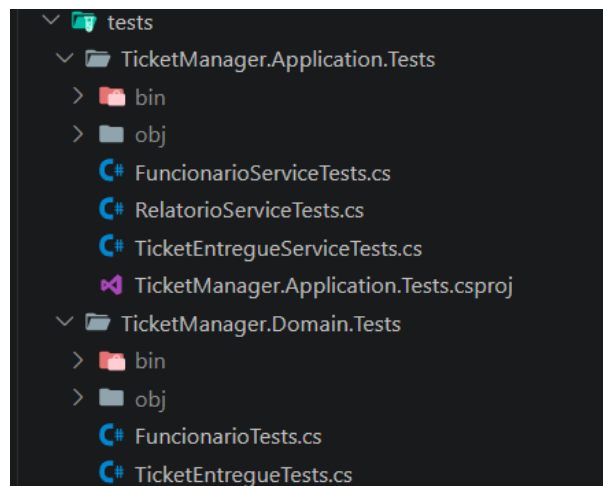
2.5.7 Executando Testes Automatizados

Os testes automatizados das camadas de Domínio e Aplicação estão localizados na pasta `tests/`. Para executá-los, basta rodar o seguinte comando na raiz do projeto:

```
dotnet test
```

A figura a seguir apresenta a estrutura de pastas dos testes, onde estão implementados os testes das camadas de Domínio e Aplicação.

Figura 4 – Estrutura de pastas dos testes automatizados



Fonte: O autor (2026)

3

Casos de Uso

Esta seção demonstra todas as funcionalidades que o usuário responsável por gerenciar o sistema pode realizar. A figura a seguir apresenta o menu principal do sistema, ponto de entrada para todas as operações descritas neste capítulo.

Figura 5 – Menu principal do sistema

```
○ ===== Ticket Manager =====  
[1] -> Cadastrar funcionário  
[2] -> Editar funcionário  
[3] -> Listar funcionários  
[4] -> Registrar ticket entregue  
[5] -> Editar ticket entregue  
[6] -> Emitir Relatórios  
[0] -> Sair  
=====
```

Escolha uma opção:

Fonte: O autor (2026)

3.1 Caso de Uso: Cadastrar um Funcionário

O usuário pode cadastrar um funcionário por vez no sistema, informando o nome e o CPF a serem cadastrados.

Figura 6 – Cadastro de um funcionário

```

○ Nome: Luiz Eduardo

CPF (somente números): 12345678900

Funcionário cadastrado com Id 1.

Pressione ENTER para continuar...

```

	id	nome	cpf	situacao	data_alteracao
▶	1	Luiz Eduardo	12345678900	A	2026-06-20 19:30:51
•	NULL	NULL	NULL	NULL	NULL

(a) Operação no console

(b) Resultado no banco de dados

Fonte: O autor (2026)

3.2 Caso de Uso: Editar um Funcionário

O usuário pode editar um funcionário por vez, alterando as informações Nome, CPF e Situação do cadastro. A seleção do funcionário a ser editado pode ser feita pelo Id ou pelo CPF. A figura a seguir apresenta o submenu de edição de funcionários.

Figura 7 – Submenu de edição de funcionários

```

[1] -> Buscar pelo ID do Funcionário
[2] -> Buscar pelo CPF do Funcionário
=====

Escolha uma opção:

```

Fonte: O autor (2026)

No primeiro exemplo, o funcionário de Id 2 — inicialmente cadastrado como Marco Aurélio, CPF 98765432100, situação ativa e data de alteração em 20/06/2026 às 19:47:04 — é selecionado pelo Id e tem o nome e o CPF atualizados para Júlio César e 00011122233, respectivamente. A situação permanece ativa, e a data de alteração passa a refletir o momento da edição (20/06/2026 às 19:48:50).

Figura 8 – Edição de funcionário selecionado pelo Id

```
[1] -> Buscar pelo ID do Funcionário
[2] -> Buscar pelo CPF do Funcionário
=====

Escolha uma opção: 1

Id do funcionário: 2

Novo nome: Júlio César

Novo CPF: 00011122233

Funcionário ativo? (S/N): S

Funcionário atualizado.

Pressione ENTER para continuar...
```

(a) Operação no console

	id	nome	cpf	situacao	data_alteracao
▶	1	Luiz Eduardo	12345678900	A	2026-06-20 19:30:51
	2	Marco Aurélio	98765432100	A	2026-06-20 19:47:04
•	NULL	NULL	NULL	NULL	NULL

(b) Banco de dados antes da edição

	id	nome	cpf	situacao	data_alteracao
▶	1	Luiz Eduardo	12345678900	A	2026-06-20 19:30:51
	2	Júlio César	00011122233	A	2026-06-20 19:48:50
•	NULL	NULL	NULL	NULL	NULL

(c) Banco de dados depois da edição

Fonte: O autor (2026)

No segundo exemplo, o funcionário de Id 5 — Michael Scott, CPF 77744411100, inicialmente com situação inativa e data de alteração em 20/06/2026 às 19:58:18 — é selecionado pelo CPF. Nesta edição, o CPF é atualizado para 99966633300 e a situação passa para ativa; a data de alteração é atualizada para 20/06/2026 às 20:18:57. Este exemplo demonstra, na prática, a reativação de um cadastro previamente inativado.

Figura 9 – Edição de funcionário selecionado pelo CPF

```
[1] -> Buscar pelo ID do Funcionário
[2] -> Buscar pelo CPF do Funcionário
=====

Escolha uma opção: 2

CPF do funcionário: 77744411100

Novo nome: Michael Scott

Novo CPF: 99966633300

Funcionário ativo? (S/N): S

Funcionário atualizado.

Pressione ENTER para continuar...[]
```

(a) Operação no console

	id	nome	cpf	situacao	data_alteracao
►	1	Luiz Eduardo	12345678900	A	2026-06-20 19:30:51
	2	Júlio César	00011122233	A	2026-06-20 19:48:50
	3	Marco Aurélio	99988877766	I	2026-06-20 19:57:30
	4	Ragnar Lodbrok	44455566677	I	2026-06-20 19:57:57
	5	Michael Scott	77744411100	I	2026-06-20 19:58:18
*	NULL	NULL	NULL	NULL	NULL

(b) Banco de dados antes da edição

	id	nome	cpf	situacao	data_alteracao
►	1	Luiz Eduardo	12345678900	A	2026-06-20 19:30:51
	2	Júlio César	00011122233	A	2026-06-20 19:48:50
	3	Marco Aurélio	99988877766	I	2026-06-20 19:57:30
	4	Ragnar Lodbrok	44455566677	I	2026-06-20 19:57:57
	5	Michael Scott	99966633300	A	2026-06-20 20:18:57
*	NULL	NULL	NULL	NULL	NULL

(c) Banco de dados depois da edição

Fonte: O autor (2026)

3.3 Caso de Uso: Listar Funcionários

O usuário pode listar os funcionários cadastrados no sistema de três formas: todos os funcionários, apenas os ativos, ou apenas os inativos. Em qualquer uma das opções, são exibidas somente as informações do próprio funcionário — nenhum dado de tickets é apresentado nesta tela.

Figura 10 – Submenu de listagem e banco de dados utilizado nos exemplos

```
[1] -> Listar Funcinários Ativos
[2] -> Editar Funcionários Inativos
[3] -> Listar Todos os funcionários

Escolha uma opção: █
```

(a) Submenu de listagem

id	nome	cpf	situacao	data_alteracao
1	Luiz Eduardo	12345678900	A	2026-06-20 19:30:51
2	Júlio César	00011122233	A	2026-06-20 19:48:50
3	Marco Aurélio	99988877766	I	2026-06-20 19:57:30
4	Ragnar Lodbrok	44455566677	I	2026-06-20 19:57:57
5	Michael Scott	77744411100	I	2026-06-20 19:58:18
*	NULL	NULL	NULL	NULL

(b) Funcionários ativos e inativos no banco

Fonte: O autor (2026)

3.3.1 Listar Todos os Funcionários Ativos

Lista todos os funcionários cuja situação esteja marcada como A (ativo) no banco de dados.

Figura 11 – Listagem de funcionários ativos

```
[1] -> Listar Funcinários Ativos
[2] -> Editar Funcionários Inativos
[3] -> Listar Todos os funcionários

Escolha uma opção: 1

ID      NOME                CPF      SITUAÇÃO
-----
1       Luiz Eduardo         12345678900  Ativo
2       Júlio César          00011122233  Ativo

Pressione ENTER para continuar... █
```

Fonte: O autor (2026)

3.3.2 Listar Todos os Funcionários Inativos

Lista todos os funcionários cuja situação esteja marcada como I (inativo) no banco de dados.

Figura 12 – Listagem de funcionários inativos

```
[1] -> Listar Funcinários Ativos
[2] -> Editar Funcionários Inativos
[3] -> Listar Todos os funcionários

Escolha uma opção: 2

ID      NOME                CPF      SITUAÇÃO
-----
3      Marco Aurélio          99988877766      Inativo
4      Ragnar Lodbrok         44455566677      Inativo
5      Michael Scott          77744411100      Inativo

Pressione ENTER para continuar...█
```

Fonte: O autor (2026)

3.3.3 Listar Todos os Funcionários do Sistema

Lista todos os funcionários cadastrados no banco de dados, independentemente da situação.

Figura 13 – Listagem de todos os funcionários

```
[1] -> Listar Funcinários Ativos
[2] -> Editar Funcionários Inativos
[3] -> Listar Todos os funcionários

Escolha uma opção: 3

ID      NOME                CPF      SITUAÇÃO
-----
1      Luiz Eduardo          12345678900      Ativo
2      Júlio César           00011122233      Ativo
3      Marco Aurélio          99988877766      Inativo
4      Ragnar Lodbrok         44455566677      Inativo
5      Michael Scott          77744411100      Inativo

Pressione ENTER para continuar...█
```

Fonte: O autor (2026)

3.4 Caso de Uso: Registrar Tickets Entregues

O usuário pode registrar a entrega de mais de um ticket por vez, sempre vinculados a um único funcionário, que pode ser selecionado pelo Id ou pelo CPF.

Figura 14 – Registro de tickets entregues

```
[1] -> Buscar pelo ID do Funcionário
[2] -> Buscar pelo CPF do Funcionário
=====

Escolha uma opção: 1

Id do funcionário: 1

Quantidade de tickets: 2

Ticket registrado com Id 1.

Pressione ENTER para continuar...
```

(a) Registro selecionando o funcionário pelo Id

```
[1] -> Buscar pelo ID do Funcionário
[2] -> Buscar pelo CPF do Funcionário
=====

Escolha uma opção: 2

CPF do funcionário: 00011122233

Quantidade de tickets: 4

Ticket registrado com Id 2.

Pressione ENTER para continuar...
```

(b) Registro selecionando o funcionário pelo CPF

	id	funcionario_id	quantidade	situacao	data_entrega
▶	1	1	2	A	2026-06-20 20:22:36
	2	2	4	A	2026-06-20 20:23:37
*	NULL	NULL	NULL	NULL	NULL

(c) Resultado no banco de dados

Fonte: O autor (2026)

3.5 Caso de Uso: Editar Tickets Entregues

O usuário pode editar, por vez, os tickets entregues a um único funcionário, alterando a quantidade de tickets e a situação do registro no banco de dados.

No exemplo a seguir, o ticket de Id 3 foi registrado, por engano, com quantidade 1000. A correção é feita editando a quantidade para 2, mantendo a situação ativa.

Figura 15 – Correção da quantidade de um ticket entregue

	id	funcionario_id	quantidade	situacao	data_entrega
▶	1	1	2	A	2026-06-20 20:22:36
	2	2	4	A	2026-06-20 20:23:37
	3	4	1000	A	2026-06-20 20:26:43
•	NULL	NULL	NULL	NULL	NULL

(a) Banco de dados antes da correção (quantidade incorreta)

```

Id do ticket: 3

Nova quantidade: 2

Ticket ativo? (S/N): S

Ticket atualizado.

Pressione ENTER para continuar...

```

(b) Operação de edição da quantidade

	id	funcionario_id	quantidade	situacao	data_entrega
▶	1	1	2	A	2026-06-20 20:22:36
	2	2	4	A	2026-06-20 20:23:37
	3	4	2	A	2026-06-20 20:26:43
•	NULL	NULL	NULL	NULL	NULL

(c) Banco de dados depois da correção

Fonte: O autor (2026)

3.6 Caso de Uso: Emitir Relatórios

O usuário pode emitir três tipos de relatório:

- Tickets entregues a um funcionário específico;
- Relatório completo por período;
- Relatório completo de todo o sistema.

Figura 16 – Submenu de relatórios e banco de dados utilizado nos exemplos

```
[1] -> Emitir tickets de um funcionário
[2] -> Emitir relatório completo por período (Funcionários + Total de Tickets + Total Geral)
[3] -> Emitir relatório completo (Funcionários + Total de tickets + Total Geral)
=====
Escolha uma opção: [ ]
```

(a) Submenu de relatórios

funcionario_id	nome	situacao	total_tickets	tickets
1	Luiz Eduardo	A	2	Id: 1 (Qtd: 2, Situação: A)
2	Júlio César	A	4	Id: 2 (Qtd: 4, Situação: A)
3	Marco Aurélio	I	NULL	NULL
4	Ragnar Lodbrok	A	2	Id: 3 (Qtd: 2, Situação: A)
5	Michael Scott	A	NULL	NULL

(b) Funcionários e respectivos tickets no banco

Fonte: O autor (2026)

3.6.1 Emitir Todos os Tickets de um Funcionário

O usuário pode emitir todos os tickets de um único funcionário por vez, selecionando-o pelo Id ou pelo CPF.

Figura 17 – Relatório de tickets de um funcionário

```
[1] -> Buscar pelo ID do Funcionário
[2] -> Buscar pelo CPF do Funcionário
=====
Escolha uma opção: 1
Id do funcionário: 1
Exibindo Tickets Entregues do Funcionário: Luiz Eduardo
ID  QUANTIDADE  SITUAÇÃO  DATA ENTREGA
-----
1  2             Ativo     20/06/2026 20:22:36
Pressione ENTER para continuar... [ ]
```

(a) Seleção pelo Id

```
[1] -> Buscar pelo ID do Funcionário
[2] -> Buscar pelo CPF do Funcionário
=====
Escolha uma opção: 2
CPF do funcionário: 00011122233
Exibindo Tickets Entregues do Funcionário: Júlio César
ID  QUANTIDADE  SITUAÇÃO  DATA ENTREGA
-----
2  4             Ativo     20/06/2026 20:23:37
Pressione ENTER para continuar... [ ]
```

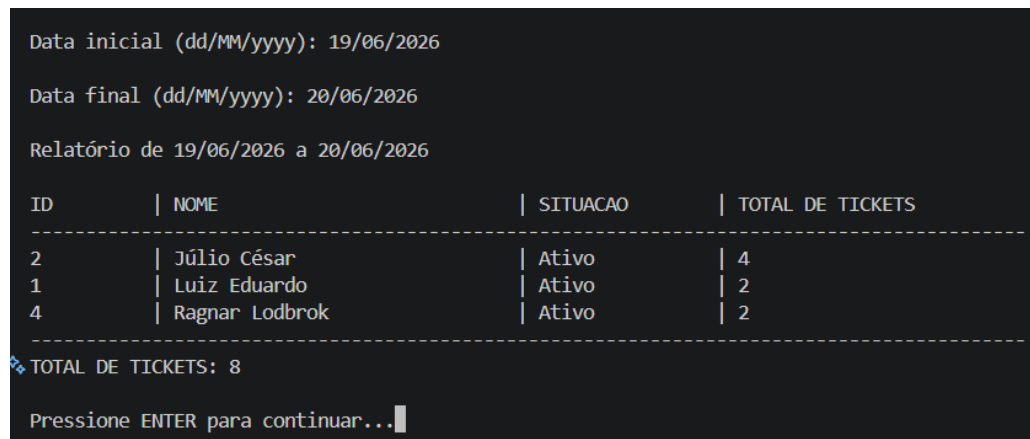
(b) Seleção pelo CPF

Fonte: O autor (2026)

3.6.2 Emitir Relatório Completo por Período

O usuário pode emitir um relatório informando uma data inicial e uma data final. O relatório exibe todos os funcionários do sistema (Id, nome e situação cadastral), o total de tickets entregues a cada um deles e, por fim, o total geral de tickets entregues no sistema dentro do período informado.

Figura 18 – Relatório completo por período



```
Data inicial (dd/MM/yyyy): 19/06/2026
Data final (dd/MM/yyyy): 20/06/2026
Relatório de 19/06/2026 a 20/06/2026
```

ID	NOME	SITUACAO	TOTAL DE TICKETS
2	Júlio César	Ativo	4
1	Luiz Eduardo	Ativo	2
4	Ragnar Lodbrok	Ativo	2

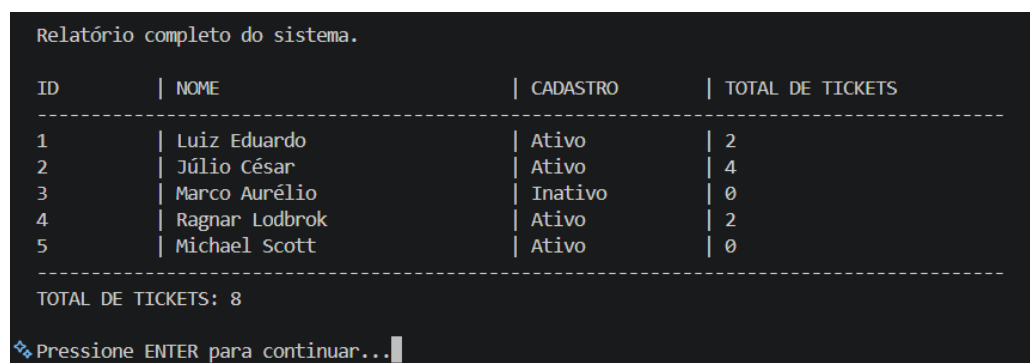
```
❖ TOTAL DE TICKETS: 8
Pressione ENTER para continuar...
```

Fonte: O autor (2026)

3.6.3 Emitir Relatório Completo de Todo o Sistema

O usuário também pode emitir um relatório completo do sistema, sem informar um período. As informações exibidas são as mesmas do relatório anterior — Id, nome e situação cadastral de cada funcionário, o total de tickets entregues a cada um e o total geral — mas considerando a totalidade dos registros, sem qualquer filtro de data.

Figura 19 – Relatório completo do sistema



```
Relatório completo do sistema.
```

ID	NOME	CADASTRO	TOTAL DE TICKETS
1	Luiz Eduardo	Ativo	2
2	Júlio César	Ativo	4
3	Marco Aurélio	Inativo	0
4	Ragnar Lodbrok	Ativo	2
5	Michael Scott	Ativo	0

```
TOTAL DE TICKETS: 8
❖ Pressione ENTER para continuar...
```

Fonte: O autor (2026)

3.7 Caso de Uso: Encerrar Programa

O usuário pode encerrar o programa a partir do menu principal, selecionando a opção correspondente. O programa é então finalizado automaticamente.

Figura 20 – Encerramento do programa

```
===== Ticket Manager =====
[1] -> Cadastrar funcionário
[2] -> Editar funcionário
[3] -> Listar funcionários
[4] -> Registrar ticket entregue
[5] -> Editar ticket entregue
[6] -> Emitir Relatórios
[0] -> Sair
=====

Escolha uma opção: 0

Pressione ENTER para continuar...
PS C:\Users\Eduar\OneDrive\Desktop\Ticket_Manager> 
```

Fonte: O autor (2026)

4

Considerações Finais e Próximos Passos

4.1 Próximos Passos

Esta seção apresenta os próximos passos planejados para a evolução deste projeto.

4.1.1 Desacoplamento das Funcionalidades de Edição

Realizar uma revisão sistemática das funcionalidades existentes, buscando identificar pontos fortemente acoplados que poderiam ser subdivididos em funcionalidades mais específicas. Por exemplo, atualmente a edição de um funcionário exige informar novamente todos os seus dados, mesmo que o usuário queira alterar apenas um deles — para alterar somente o nome, por exemplo, é necessário informar também o CPF e a situação do cadastro. O ideal seria desacoplar essa funcionalidade em operações mais específicas: uma opção para alterar apenas o nome, outra para alterar apenas o CPF, outra para alterar apenas a situação e, por fim, uma opção que permita alterar todos os campos de uma vez.

4.1.2 Filtros Personalizados para Relatórios

Implementar uma funcionalidade que permita a emissão de relatórios por meio de filtros personalizados. Por exemplo: listar todos os funcionários, dentro de um intervalo de datas, que estejam inativos e possuam mais de um ticket entregue — nesse único exemplo já são combinadas três variáveis de filtro (intervalo de tempo, situação cadastral e quantidade de tickets).

4.1.3 Interface Web

Substituir o console por uma interface web, permitindo que o sistema se torne escalável e possa ser utilizado como uma solução real em algum contexto profissional que demande esse tipo de serviço.